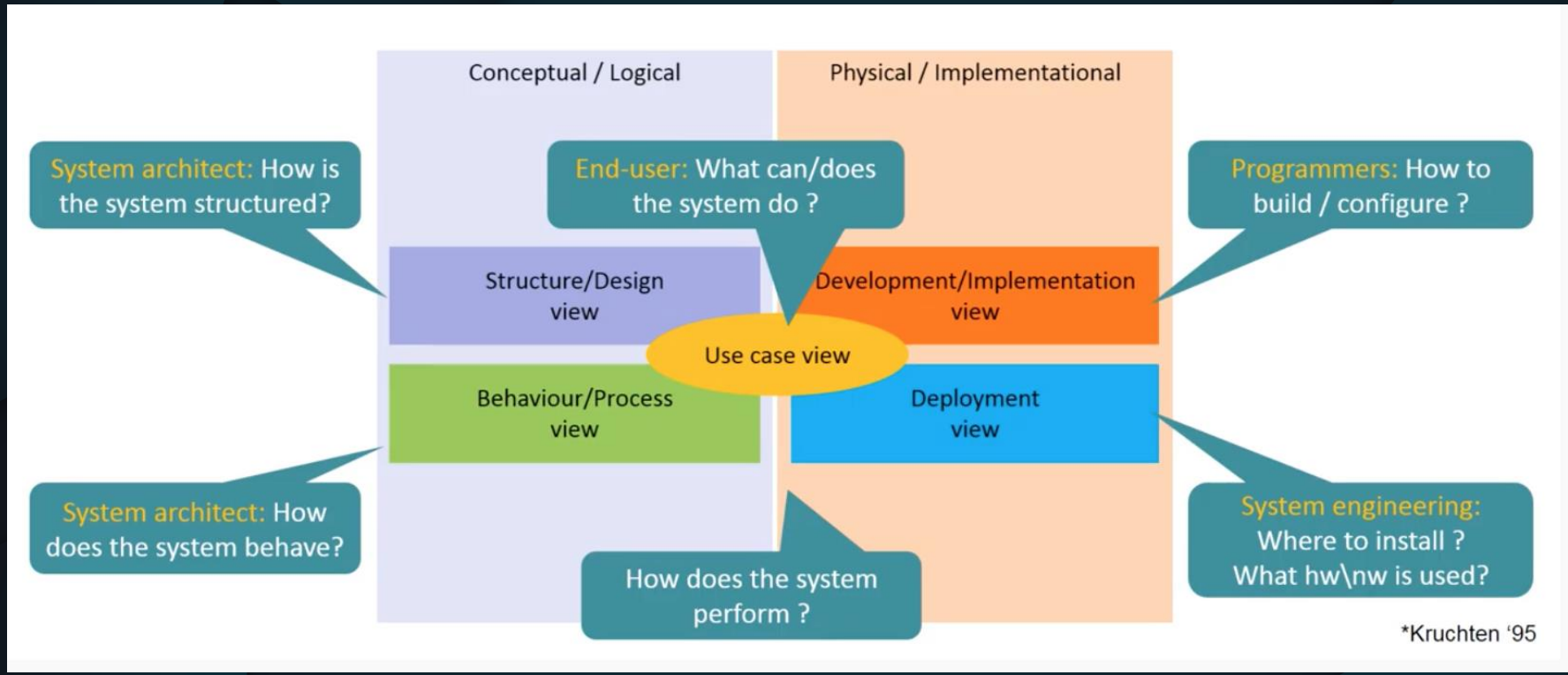
Fundamental Principles of Object-Oriented Analysis (Use Cases)

Katerina Mangaroska week 36


Conceptual / Logical
Physical / Implementational
System architect: How is the system structured?
End-user: What can/does the system do ?
Programmers: How to build / configure ?
Structure/Design view
Development/Implementation view
Use case view
Behaviour/Process view
Deployment view
System architect: How does the system behave?
How does the system perform ?
System engineering: Where to install ? What hw\nw is used?
*Kruchten '95

01

Use cases and use-case diagrams

The **use of a system** is described through use cases. Use cases describe interactions between actors and the system itself. They are used to **scope the system** and identify which interfaces there should be.

Two primary aspects of functional requirements:

A **use case** is an activity the system performs in response to a request by a user.

A **domain model** captures the most important types of objects in the context of the business. The domain model represents the ‘things’ that exist or events that transpire in the business environment. - Ivar Jacobsen

How to identify a use case?

I. User goal technique - ask users to describe their goals for using the new or updated system.

- what the system could do to improve the user's performance and productivity?
- streamlining tasks the user currently performs or enabling the user to perform new task.

--> User stories will help analysts to identify and define use cases.

* CRUD (*Create*, *Read* or *Report*, *Update*, and *Delete*) technique

an approach in which an analyst looks at each type of data and includes use cases that create the data, read or report on the data, update the data, and delete the data.

System analysis activities

User goal technique

User/actor	User goal and resulting use case
Order clerk	Look up item availability Create new order Update order
Shipping clerk	Record order fulfillment Record back order
Merchandising manager	Create special promotion Produce catalog activity report

CRUD technique

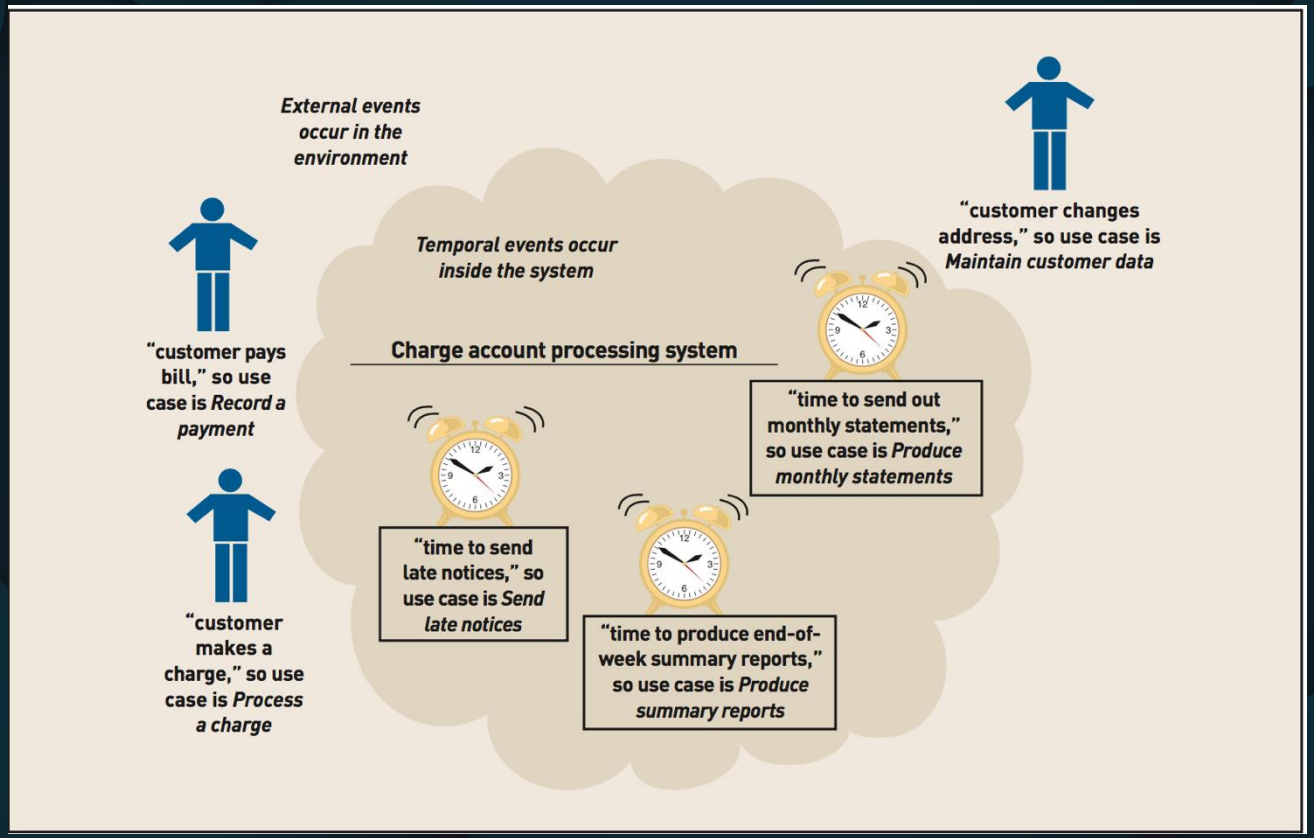
Data entity/class	CRUD	Resulting use case
Customer	Create	Add new customer
	Read/Report	Look up customer Produce customer list List customer orders
	Update	Update customer information
	Delete	Delete inactive customer

How to identify a use case?

II. event decomposition technique - identify all business events the information system responds to, with each event leading to a use case. An event occurs at a specific time and place, can be described, and should be remembered by the system.

- typing in a customer name on a form --> too narrow to be useful
- the entire process of adding a new customer
- working with customers all day (adding new customers, updating customer records, deleting customers, following up on late-paying customers) --> too broad to be useful

Event decomposition technique



Use case

The appropriate level of detail for identifying use cases is one that focuses on **elementary business processes (EBPs)**.

Elementary business process - a task that is performed by one person in one place in response to a business event, adds measurable business value, and leaves the system and its data in a stable and consistent state.

e.g., fill shopping cart - there is one person filling the cart, and there is measurable value for the customer as items are added to the cart.

USE CASE = VERB + NOUN

Examples of use cases

Use case	Brief use case description
Create customer account	User/actor enters new customer account data, and the system assigns account number, creates a customer record, and creates an account record.
Look up customer	User/actor enters customer account number, and the system retrieves and displays customer and account data.
Process account adjustment	User/actor enters order number, and the system retrieves customer and order data; actor enters adjustment amount, and the system creates a transaction record for the adjustment.

© Cengage Learning®

Use case diagram

Captures system functionality as seen by the users. (user-focused)

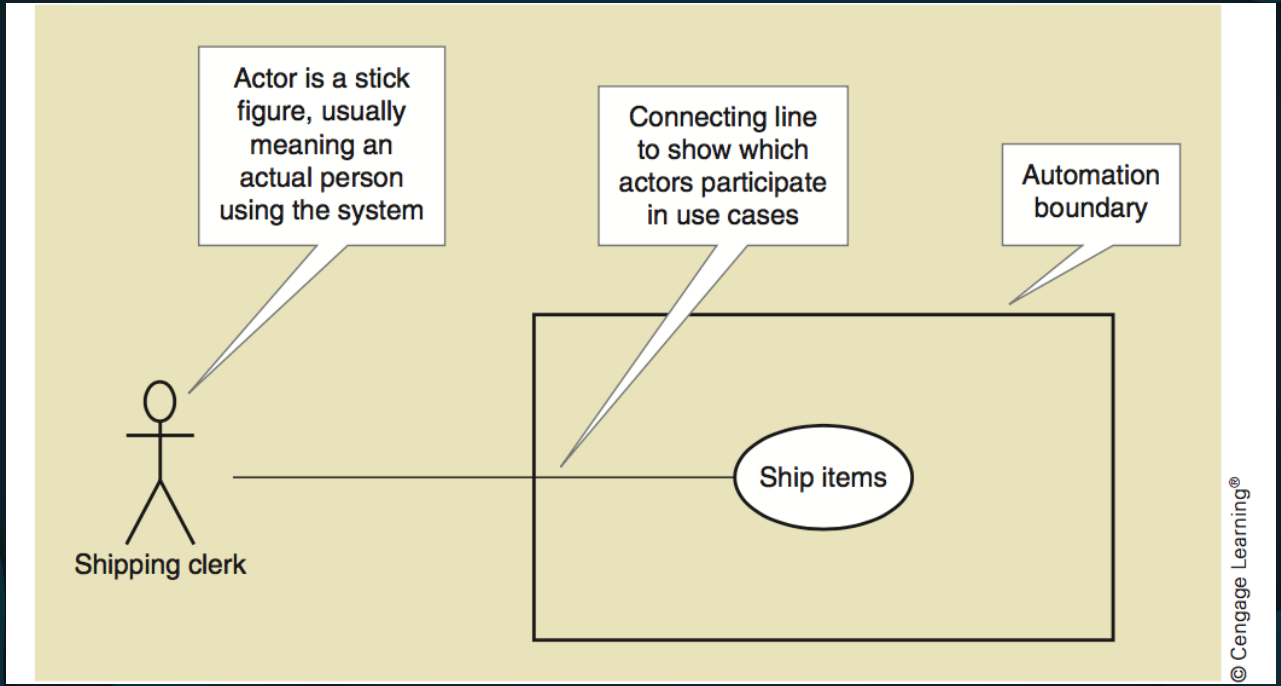
Use case diagram is a diagram to show the various user roles and how those roles use the system.

Developed in early stages of the project to document the system activities.
The **use case diagram** is the UML model used to illustrate use cases and their relationship to users.

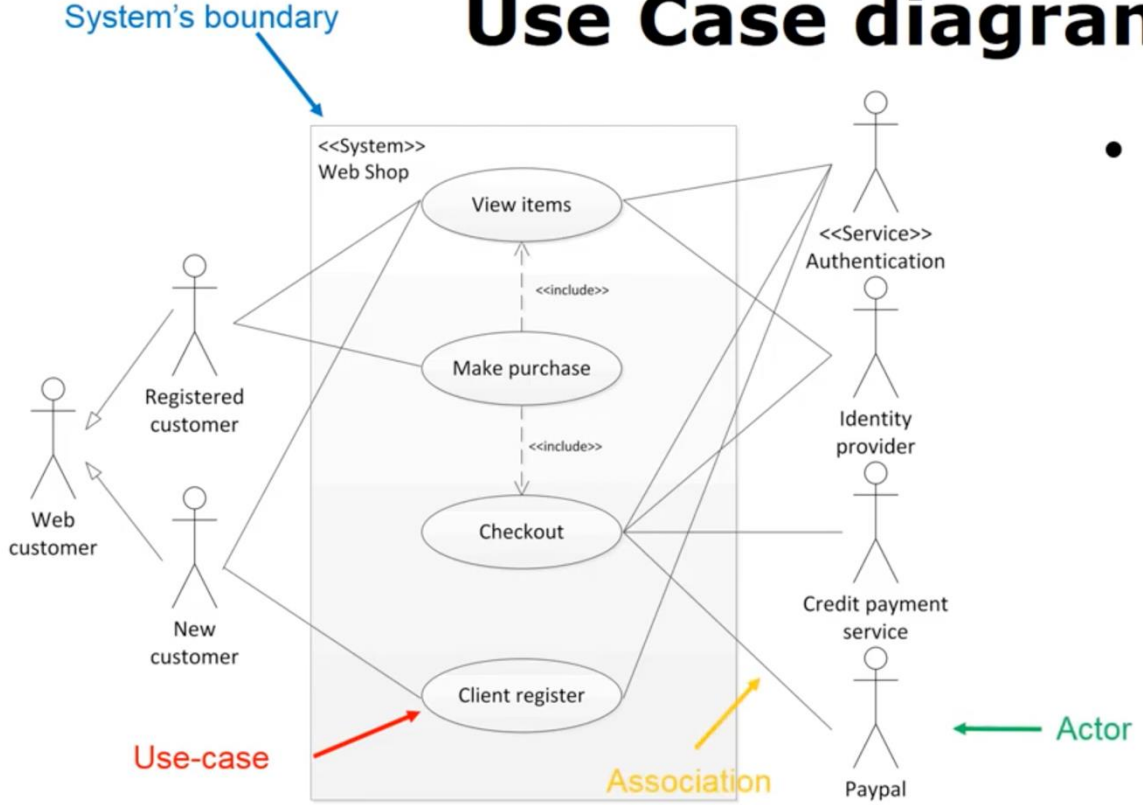
Purpose:

- specify the context of a system
- capture the requirements of a system
- validate an architecture completeness
- drive implementation and generate test cases

Use case diagram



Use Case diagram

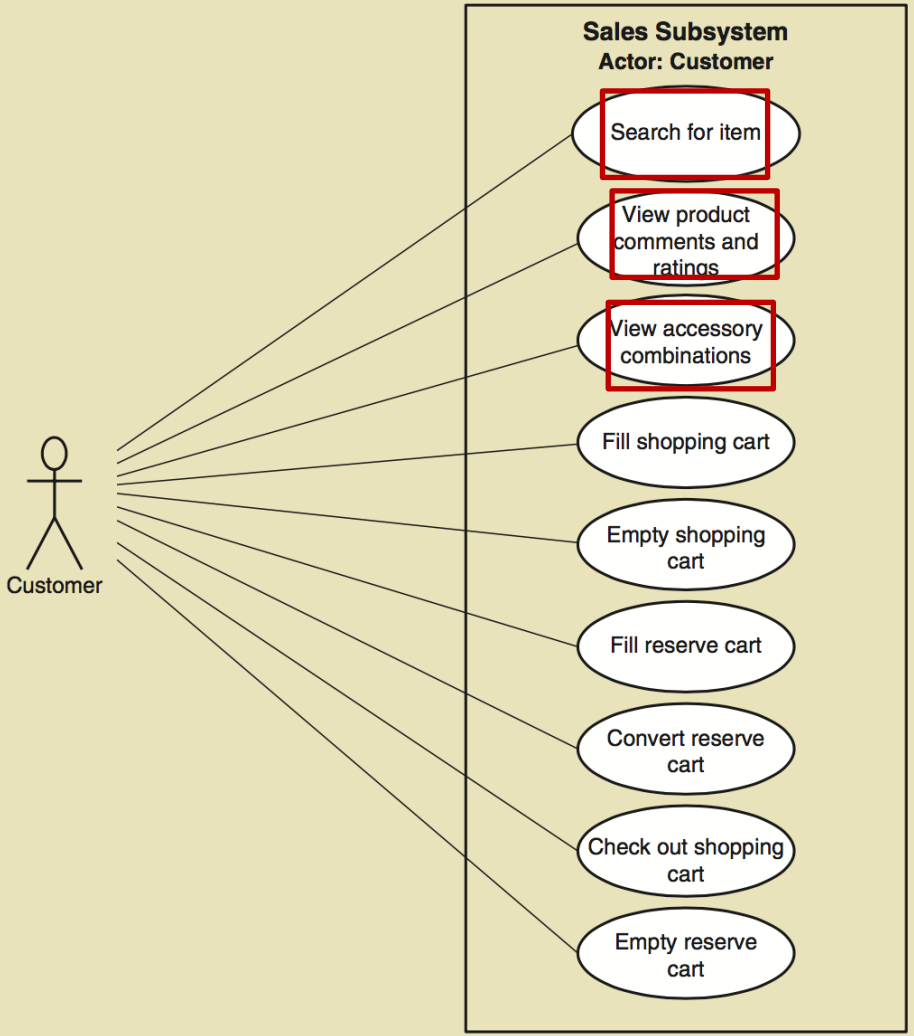


- Basic elements:
  - Actors
  - Use-cases
  - Associations
  - System's boundary

Use-case diagram for a particular actor

List of use-cases

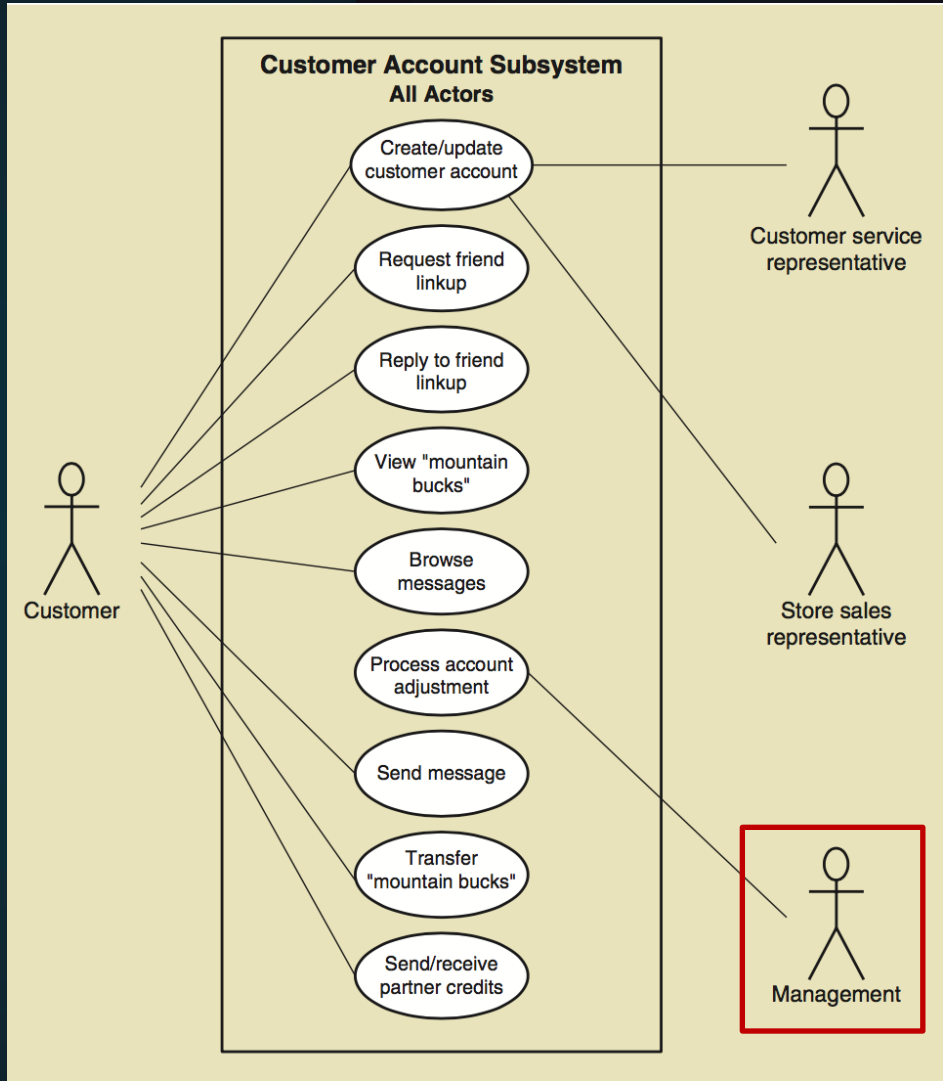
CSMS Sales Subsystem	
Use cases	**Users/actors**
Search for item	Customer, customer service representative, store sales representative
View product comments and ratings	Customer, customer service representative, store sales representative
View accessory combinations	Customer, customer service representative, store sales representative
Fill shopping cart	Customer
Empty shopping cart	Customer
Check out shopping cart	Customer
Fill reserve cart	Customer
Empty reserve cart	Customer
Convert reserve cart	Customer
Create phone sale	Customer service representative
Create store sale	Store sales representative



Use-case diagram for all actors

List of use-cases

CSMS Customer Account Subsystem	
Use cases	**Users/actors**
Create/update customer account	Customer, customer service representative, store sales representative
Process account adjustment	Management
Send message	Customer
Browse messages	Customer
Request friend linkup	Customer
Reply to linkup request	Customer
Send/receive partner credits	Customer
View "mountain bucks"	Customer
Transfer "mountain bucks"	Customer

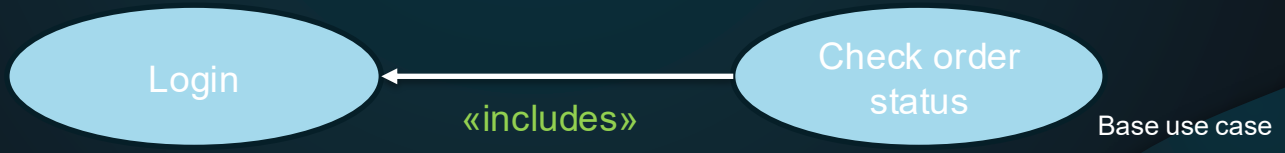


Include and extend use-case relationships

An include relationship is a relationship in which one use case (the base use case) includes the functionality of another use case (the inclusion use case). *The base use case is incomplete without the included use case.* The included use case is mandatory and not optional.

When to use it?

- The behavior of the inclusion use case is common to two or more use cases.
- The result of the behavior that the inclusion use case specifies, not the behavior itself, is important to the base use case.



* The solid line needs to be a dashed line

Include and extend use-case relationships

An extend relationship is a relationship in which one use case (extension) extends the behavior of another use case (base). *It extends the base use case and adds more functionality to the system.* The extension use case owns the extend relationship. You can specify several extend relationships for a single base use case.

When to use it?

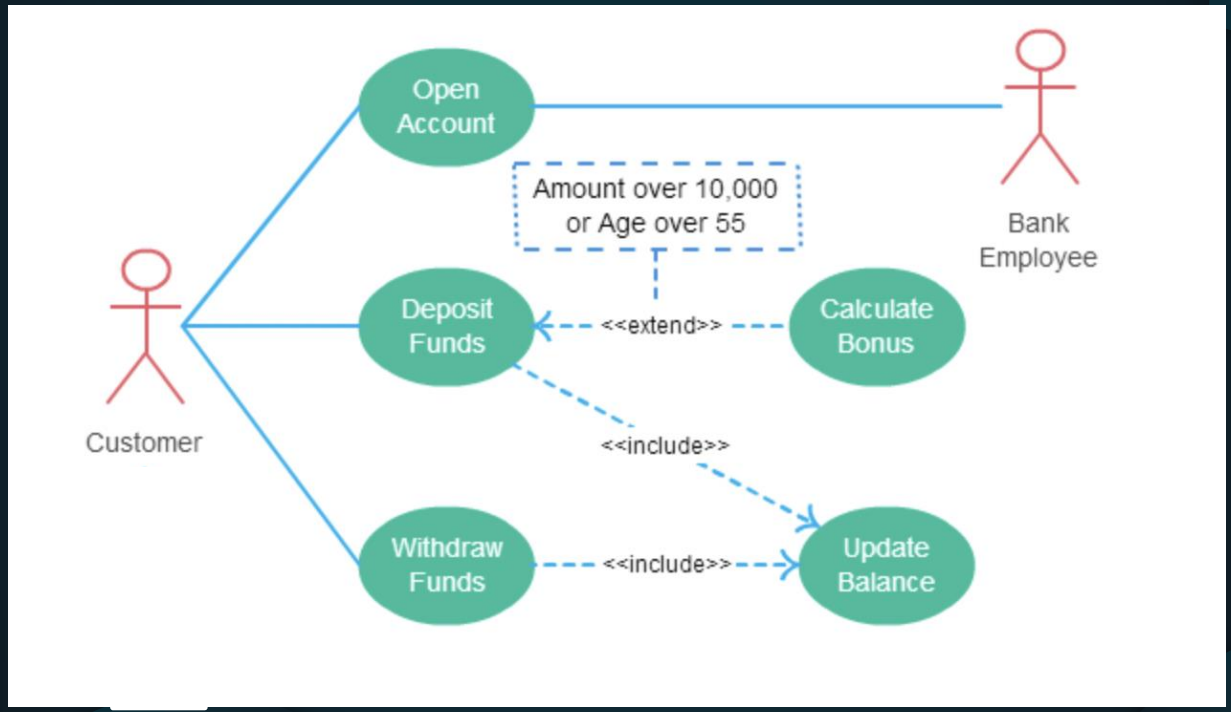
- A part of a use case that is optional system behavior.
- A subflow is executed only under certain conditions.
- A set of behavior segments that may be inserted in a base use case.



* The solid line needs to be a dashed line

Include and extend use-case relationships

Example



The extended use case must be meaningful on its own!

Use case descriptions

Levels of detail:

1. Brief use case descriptions
   - enough detail for very simple use cases
2. Fully developed use case descriptions
   - To create a comprehensive, robust system that truly meets users' needs, you must understand the detailed steps of each use case.

Use case	Brief use case description
Create customer account	User/actor enters new customer account data, and the system assigns account number, creates a customer record, and creates an account record.
Look up customer	User/actor enters customer account number, and the system retrieves and displays customer and account data.
Process account adjustment	User/actor enters order number, and the system retrieves customer and order data; actor enters adjustment amount, and the system creates a transaction record for the adjustment.

Actor - a separate flow of activities depending on which actor invokes the use case.

- customer service representative creating account over the phone/online chat.

Stakeholders - who are interested parties other than specific actors who invoked the use case.

- accounting - accurately capturing billing and credit card information.
- marketing - perform statistical analysis and create marketing promotions

Use case name:	*Create customer account*.	
Scenario:	Create online customer account.	
Triggering event:	New customer wants to set up account online.	
Brief description:	Online customer creates customer account by entering basic information and then following up with one or more addresses and a credit or debit card.	
Actors:	Customer.	
Related use cases:	Might be invoked by the *Check out shopping cart* use case.	
Stakeholders:	Accounting, Marketing, Sales.	
Preconditions:	Customer Account subsystem must be available. Credit/debit authorization services must be available.	
Postconditions:	Customer must be created and saved. One or more Addresses must be created and saved. Credit/debit card information must be validated. Account must be created and saved. Address and Account must be associated with Customer.	
Flow of activities:	**Actor**	**System**
	1. Customer indicates desire to create customer account and enters basic customer information.	1.1 System creates a new customer. 1.2 System prompts for customer addresses.
	2. Customer enters one or more addresses.	2.1 System creates addresses. 2.2 System prompts for credit/debit card.
	3. Customer enters credit/debit card information.	3.1 System creates account. 3.2 System verifies authorization for credit/debit card. 3.3 System associates customer, address, and account. 3.4 System returns valid customer account details.
Exception conditions:	1.1 Basic customer data are incomplete. 2.1 The address isn't valid. 3.2 Credit/debit information isn't valid.	

Pre-conditions identify what the state of the system must be for the use case to begin.

- what objects must already exist, what information must be available.

Post-conditions identify what must be true upon completion of the use case.

- what new objects are created or updated by the use case and how objects need to be associated.

- testing the use case after it is implemented;
- objects involved in the use case that are important for design.

Use case name:	*Create customer account*.
Scenario:	Create online customer account.
Triggering event:	New customer wants to set up account online.
Brief description:	Online customer creates customer account by entering basic information and then following up with one or more addresses and a credit or debit card.
Actors:	Customer.
Related use cases:	Might be invoked by the *Check out shopping cart* use case.
Stakeholders:	Accounting, Marketing, Sales.
Preconditions:	Customer Account subsystem must be available. Credit/debit authorization services must be available.
Postconditions:	Customer must be created and saved. One or more Addresses must be created and saved. Credit/debit card information must be validated. Account must be created and saved. Address and Account must be associated with Customer.
Flow of activities:	**Actor** / **System**
	1. Customer indicates desire to create customer account and enters basic customer information. 2. Customer enters one or more addresses. 3. Customer enters credit/debit card information.
	1.1 System creates a new customer. 1.2 System prompts for customer addresses. 2.1 System creates addresses. 2.2 System prompts for credit/debit card. 3.1 System creates account. 3.2 System verifies authorization for credit/debit card. 3.3 System associates customer, address, and account. 3.4 System returns valid customer account details.
Exception conditions:	1.1 Basic customer data are incomplete. 2.1 The address isn't valid. 3.2 Credit/debit information isn't valid.

Example

An example to look at and study at home.

- Ship items use case

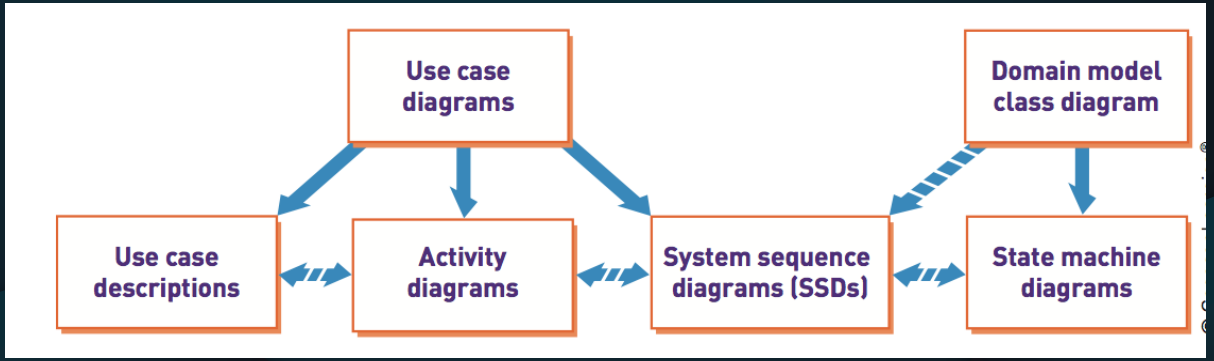
Use case name:	*Ship items*.	
Scenario:	Ship items for a new sale.	
Triggering event:	Shipping is notified of a new sale to be shipped.	
Brief description:	Shipping retrieves sale details, finds each item and records it is shipped, records which items are not available, and sends shipment.	
Actors:	Shipping clerk.	
Related use cases	None.	
Stakeholders:	Sales, Marketing, Shipping, warehouse manager.	
Preconditions:	Customer and address must exist. Sale must exist. Sale items must exist.	
Postconditions:	Shipment is created and associated with shipper. Shipped sale items are updated as shipped and associated with the shipment. Unshipped items are marked as on back order. Shipping label is verified and produced.	
Flow of activities:	**Actor**	**System**
	1. Shipping requests sale and sale item information.	1.1 System looks up sale and returns customer, address, sale, and sales item information.
	2. Shipping assigns shipper.	2.1 System creates shipment and associates it with the shipper.
	3. For each available item, shipping records item is shipped.	3.1 System updates sale item as shipped and associates it with shipment.
	4. For each unavailable item, shipping records back order.	4.1 System updates sale item as on back order.
	5. Shipping requests shipping label supplying package size and weight.	5.1 System produces shipping label for shipment. 5.2 System records shipment cost.
Exception conditions:	2.1 Shipper is not available to that location, so select another. 3.1 If order item is damaged, get new item and updated item quantity. 3.1 If item bar code isn't scanning, shipping must enter bar code manually. 5.1 If printing label isn't printing correctly, the label must be addressed manually.	

Integrating analysis models

Use case model includes use case diagrams, use case descriptions, activity diagrams, and system sequence diagrams.

Not all use cases need to be modeled in detail.

Use case diagram and the domain model class diagram are the primary models from which others draw information. You should develop those two diagrams as completely as possible.



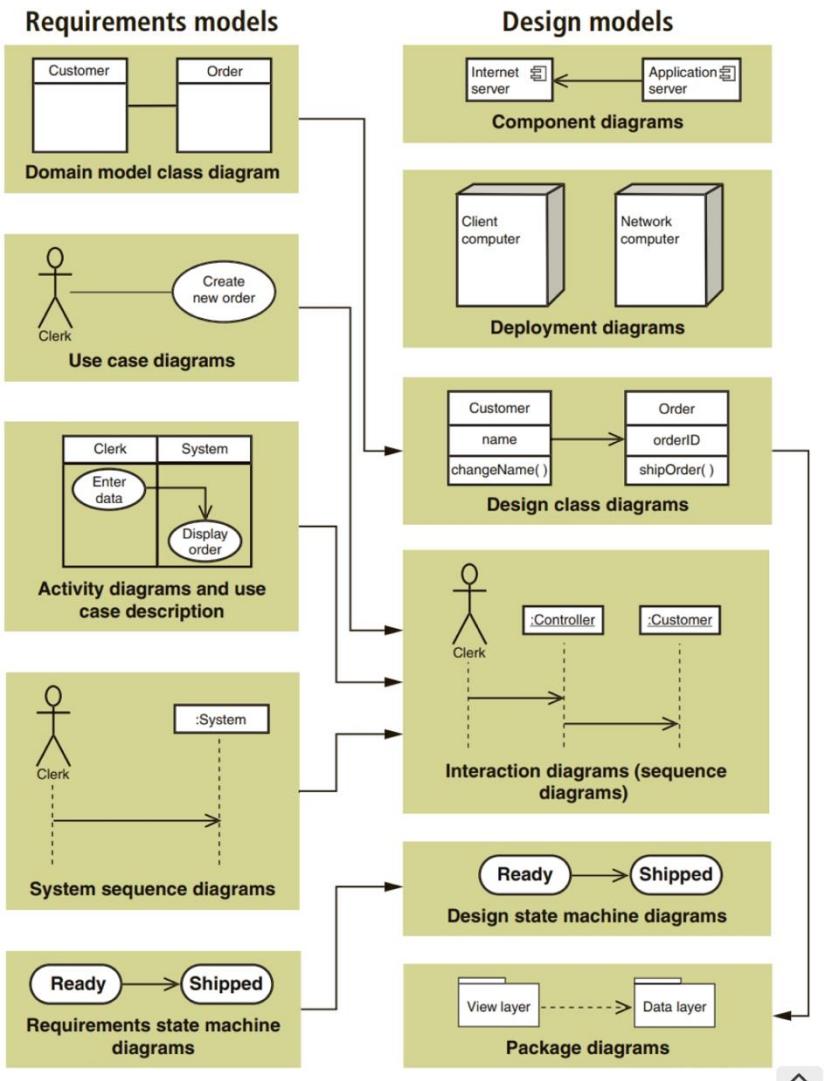
Summary

The use case diagrams identify the elementary business processes that the system needs to support—in other words, all the ways users want to use the system to carry out processing goals.

The activity diagrams and the use case descriptions document the internal workflow of each use case.

The system sequence diagram is closely related to the activity diagrams, except that they show the messages or data that are sent back and forth between the user and the system during the steps of the use case.

The domain model class diagram identifies all the classes, or “things,” that are important in the problem domain.



Does anyone have any questions?

THANK YOU!

katerina.mangaroska@usn.no